Projeto de IA: Geração e Tutoria Adaptativa para Programação Competitiva

Este documento formaliza a proposta de trabalho para a disciplina de Inteligência Artificial, englobando a apresentação do problema e o planejamento da extensão sugerida.

1. O Artigo Base (Apresentação do Problema e Solução)

A base deste projeto é fundamentada no seguinte artigo recente:

- **Título:** AutoCode: LLMs as Problem Setters for Competitive Programming
- **Autores:** Shang Zhou et al.
- **Ano:** 2025

O Problema: Criar problemas inéditos e testes rigorosos para programação competitiva exige precisão extrema para evitar soluções e atalhos lógicos errados. O artigo investiga a capacidade de usar IA para gerar problemas nesse patamar de exigência.
A Solução: O *AutoCode* implementa um protocolo de dupla verificação, gerando enunciados de nível de competição e soluções de força bruta para cruzar resultados. Isso atesta a validade matemática da geração (atingindo quase 99% de consistência com juízes oficiais).

2. O Objetivo e Extensão Proposta

Proponho modificar o foco estritamente gerador do trabalho original para criar uma plataforma de tutoria adaptativa e avaliação local, integrando a identificação de fraquezas dos estudantes.

- **Juiz Local (Mini-Judge):** O aplicativo executará os casos de teste gerados pelo *AutoCode* localmente na máquina do aluno, garantindo métricas rigorosas e determinísticas como Tempo Excedido (TLE) ou Erro de Execução (RE).
- **Tutoria Baseada em Fraquezas:** O histórico dos erros não será usado para dar a resposta correta de imediato. Ele será enviado à IA para mapear deficiências algorítmicas, gerar uma explicação pontual e construir um exercício imediato para treinar o elo fraco do aluno.

Fundamentação Científica da Extensão (Artigos Complementares)

Para comprovar cientificamente que esta abordagem proposta é possível, o projeto se sustenta em mais dois trabalhos do estado da arte:

- **Mapeamento de Fraquezas (Codigen):** O artigo *"Codigen: Enhancing Personalized*

Programming Education with Visual Authoring and LLMs" (VL/HCC, 2025) comprova a eficácia de arquiteturas LLM adaptativas que rastreiam as dificuldades do aluno e moldam os exercícios para aprimorar sua educação de forma personalizada.

- **Identificação e Recomendação (OI-Assistant):** O estudo *"OI-Assistant: A Retrieval Augmented System for Similar Problem Discovery and Interactive Learning in Competitive Programming"* (IOI Journal, 2025) valida com precisão a estrutura de recuperação de informação para prover recomendações interativas em programação competitiva, algo vital para a geração guiada sugerida no projeto.

3. Arquitetura e Execução

O planejamento técnico foca na viabilidade computacional, dividindo o motor de execução local do processamento semântico na nuvem:

Módulo	Tecnologia Proposta	Função na Plataforma
Interface Web	Python (Streamlit)	Renderizar os problemas, mostrar métricas e receber a submissão de código.
Mini-Judge (Motor)	Subprocess Isolado	Rodar os casos de teste estritamente em Python local, emitindo vereditos (Ex: TLE, WA).
Tutor API GenAI	API (Ex: Gemini)	Analisar a saída do Juiz, descobrir a falha e moldar a tutoria e testes seguintes.

Exemplo Lógico da Orquestração (Código Interno)

```
[SYSTEM PROMPT]
Você atua como um tutor assistente de Programação Competitiva.
INFORMAÇÃO DO JUIZ LOCAL: O aluno submeteu um algoritmo.
O juiz apontou: "Time Limit Exceeded no caso de teste N=100.000".
TAREFA: Avaliando que o problema requer complexidade O(N log N) e o aluno fez O(N^2),
gere um conselho didático focando em Estruturas de Dados ou Busca Binária,
e em seguida gere um problema novo focado nessa deficiência detectada.
```